Pune Institute of Computer Technology

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Department of Computer Engineering

AY : 2026-27

CNL MINI PROJECT REPORT

ON

NETWORK TRAFFIC ANALYZER & REAL-TIME ANOMALY DETECTOR (MINI INTRUSION DETECTION SYSTEM)

Submitted By

31430– Atharv Magar
31434– Aman Mirajkar
31440 -Shritej Nehere
31442 – Pranav Oswal

Class : TY Computer Engineering
Batch : L4

Under the guidance of

Prof. P.R.Patil

Index

1. Problem statement ……..2
2. Objectives ……...3
3. Methodology Approach ……..4
4. Implementation ……...7
5. Results / Outputs ……...10
6. Conclusion ……13
7. References and documentation ……14

1. Problem statement

1.1 Background and motivation

Computer networks carry a continuous stream of frames and packets generated by users, applications, servers, and network services. Visibility into this traffic helps an administrator understand protocol usage, locate high-volume hosts, investigate performance changes, and notice patterns that may indicate reconnaissance or attempted denial of service. Manual packet inspection is valuable but can require repeated filtering and interpretation, particularly when the goal is to watch traffic continuously and surface a small set of suspicious patterns automatically.

The Network Traffic Analyzer & Real-Time Anomaly Detector is a local, browser-based monitoring tool and educational mini Intrusion Detection System (IDS). It passively observes traffic on a selected interface, extracts metadata from supported Layer 2 to Layer 4 protocols, aggregates live statistics, and evaluates three deterministic rules: TCP port scanning, an unusually high rate of bare TCP SYN packets, and conflicting ARP address mappings. Its dashboard makes these observations visible without requiring the user to manually inspect every packet.

1.2 Problem definition

The project addresses the problem of providing a lightweight, explainable view of live network traffic that combines protocol classification, host-level traffic accounting, rolling bandwidth statistics, and alerts for selected suspicious packet patterns. The system must remain practical for a student lab: it should be locally runnable, testable without live packet capture, memory-bounded, and safe to demonstrate in an isolated network.

1.3 Existing approach and proposed solution

A packet analyzer such as Wireshark offers detailed protocol dissection and manual filtering. This project has a different emphasis: it converts observed packets into compact metadata, aggregates the observations into rolling metrics, and evaluates a small set of explainable heuristic rules automatically. It is not intended to replace Wireshark or provide a complete enterprise security platform. Wireshark can also be used as an independent reference during lab validation.

Aspect	Proposed project approach
Traffic acquisition	Passive Scapy capture after an explicit user action; no packet injection by the running application.
Data retained	Header metadata and derived counters only; application payloads and raw Scapy packet objects are not stored or sent to the UI.
Analysis	Deterministic sliding-window rules, transparent thresholds, and evidence attached to each alert.
Presentation	React dashboard with near-real-time summaries, graphs, host statistics, recent packet metadata, and alert acknowledgement.
Runtime state	Bounded in-memory session state; no database or long-term packet archive in the MVP.

1.4 Scope and boundaries

The implemented scope includes interface discovery and selection, explicit capture start/stop, parsing of Ethernet where available, IPv4, IPv6, TCP, UDP, ICMP, ICMPv6, ARP and fallback traffic, rolling metrics, top-host accounting, alert storage, REST endpoints, WebSocket updates, and the interactive dashboard. The MVP deliberately excludes machine-learning detection, packet payload inspection, PCAP recording, packet blocking, automatic response actions, persistent databases, user accounts, remote sensors, and public/cloud deployment. These exclusions keep the implementation focused and its outputs explainable.

2. Objectives

- Capture packets passively from a network interface selected by the user, with explicit start and stop controls.
- Parse supported Layer 2, Layer 3, and Layer 4 headers into a normalized, serializable packetobservation model.
- Classify protocols and calculate session packet/byte totals, rolling throughput, protocol distribution, and per-host TX/RX statistics.
- Detect TCP port-scan behaviour, possible SYN-rate flooding, and conflicting ARP IP-to-MAC mappings using deterministic time-window rules.
- Generate explainable alerts with timestamps, rule identifiers, supporting evidence, deduplication, cooldown handling, and acknowledgement.
- Expose capture state and analysis results through a local FastAPI REST interface and a WebSocket stream refreshed approximately once per second.
- Provide a responsive React dashboard that displays status, charts, top talkers, recent packet metadata, and alert details.
- Validate parsing, metrics, detectors, API behaviour, frontend types/build, and controlled lab scenarios while keeping the sensor passive and memory bounded.

3. Methodology Approach (Architecture diagram and flow diagram)

3.1 System architecture

The design separates packet acquisition, normalization, metrics, anomaly detection, API delivery, and presentation. The capture callback passes each packet to a defensive parser. A normalized observation is sent to both the metrics store and the detection engine. Metrics and alert snapshots are then exposed through FastAPI, while a periodic WebSocket publisher streams a compact combined snapshot to the browser. This separation makes parsing and detection independently testable without requiring root privileges or a live interface.

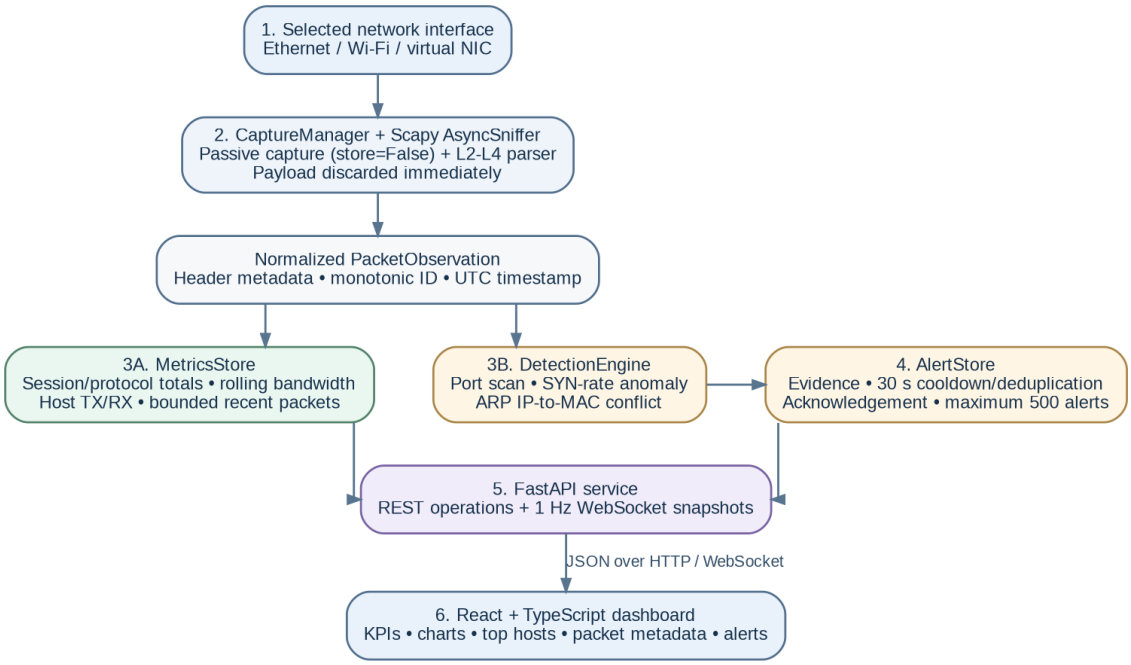


Figure 1. High-level architecture of the Network Traffic Analyzer and anomaly-detection pipeline.

3.2 End-to-end processing flow

Capture is started only after the user selects an available interface. For every observed frame, the parser extracts supported fields and produces a bounded metadata record; the system updates counters, evaluates applicable rules, and creates or updates alerts when a condition is met. The frontend does not independently infer detections: it presents the backend snapshot and sends user actions such as start, stop, and acknowledge.

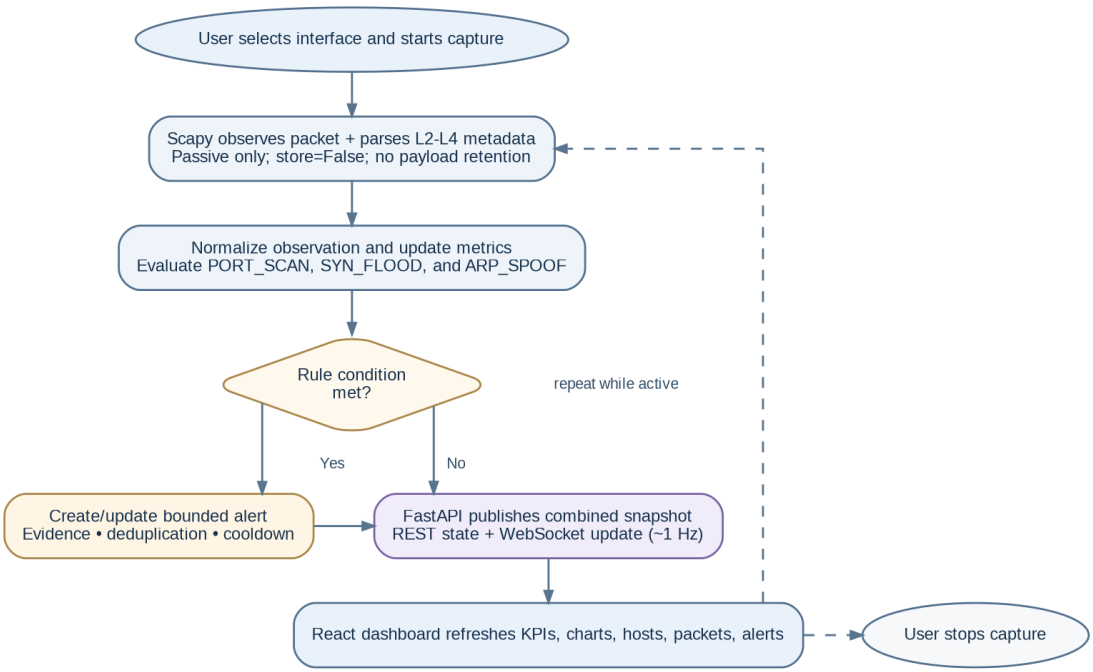


Figure 2. Packet-processing, detection, alerting, and dashboard-update workflow.

3.3 Isolated network topology for validation

Validation is intended for a private host-only network or an isolated Mininet topology. The illustrated two-VM option places the analyzer and monitored host at 192.168.56.10 and a controlled test client at 192.168.56.20 on an internal-only segment. Test traffic must remain inside this lab; public IPs, campus networks, production systems, and unrelated devices must never be targets.

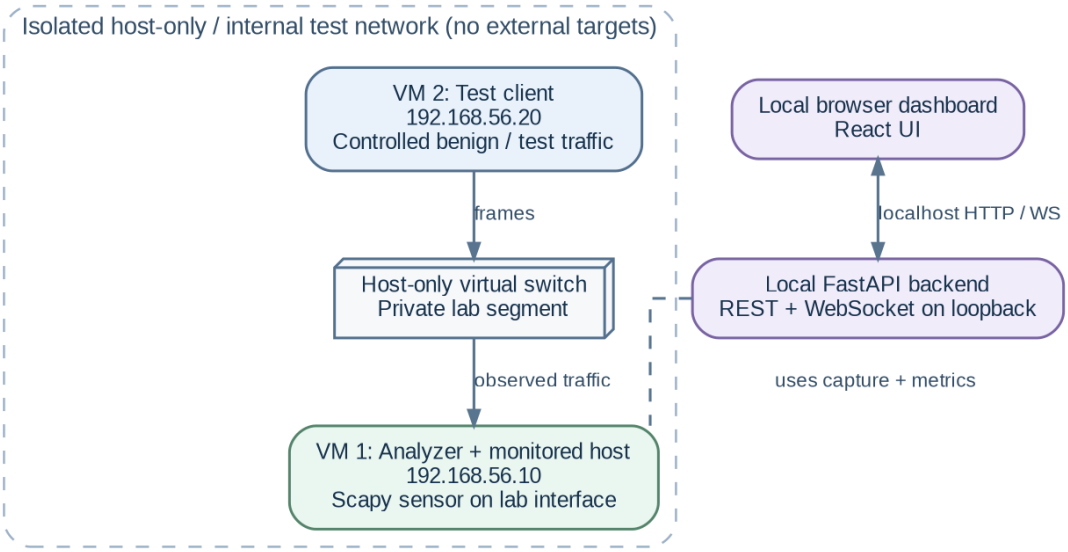


Figure 3. Example host-only virtual-machine topology for controlled demonstrations.

3.4 Networking concepts applied

Concept	Application in this project
Layer 2: Ethernet and ARP	Ethernet source/destination MAC metadata is parsed where available. ARP sender-IP/sender-MAC claims are tracked to identify conflicting mappings.
Layer 3: IPv4 and IPv6	Source/destination addresses and protocol information are extracted to group traffic by endpoint and classify packets.
Layer 4: TCP and UDP	TCP/UDP ports are tracked. TCP flags, particularly SYN and ACK, support port-scan and SYNrate heuristics.
TCP connection establishment	A bare SYN is identified as SYN set and ACK unset; a SYN by itself is not proof that a connection or attack succeeded.
Time-window analysis	Timestamped observations are retained in rolling windows so thresholds represent rates/patterns over a period rather than session totals alone.
Client-server communication	The React browser consumes JSON via REST and WebSocket from a local FastAPI backend.

4. Implementation

4.1 Technology stack

Layer	Technology / component	Role
Packet capture	Python 3.11+; Scapy AsyncSniffer	Observes packets on the selected interface with packet storage disabled.
Backend/API	FastAPI; Pydantic schemas	Validates requests and exposes health, capture, metrics, packet, alert, and WebSocket interfaces.
Analysis model	Immutable PacketObservation dataclass	Normalizes header metadata for downstream processing and deterministic tests.
Metrics and state	Thread-safe in-memory MetricsStore and AlertStore	Maintains counters, rolling rates, recent packet metadata, evidence alerts, and acknowledgement state.
Frontend	React 18; TypeScript; Vite	Provides capture controls and live operational dashboard.
Visualizations	Recharts; Lucide icons	Renders protocol distribution, traffic/bandwidth trends, status, and alert affordances.
Validation	pytest; TypeScript check; Vite build; isolated VM/Mininet lab; Wireshark reference	Checks core logic and packaging, then supports controlled end-to-end demonstrations.

4.2 Capture and packet normalization

The CaptureManager validates the chosen operating-system interface and wraps Scapy AsyncSniffer. Capture lifecycle states are tracked as stopped, starting, running, stopping, and error, and repeated start requests are handled without starting duplicate capture sessions. Scapy packet storage is disabled (store=False), so the sniffer does not accumulate captured packet objects.

The parser extracts the relevant fields from Ethernet, IPv4/IPv6, TCP/UDP, ICMP/ICMPv6, and ARP layers, and falls back to OTHER for unsupported traffic. It creates an immutable PacketObservation containing normalized addresses, ports and flags where applicable, packet/frame length, a monotonically generated packet ID, and a timezone-aware UTC timestamp. The parser is designed to handle unsupported or malformed packets defensively instead of terminating the capture callback.

A strict zero-payload policy is applied: raw application data is not retained, logged, serialized through the API, or broadcast by WebSocket. The dashboard can show recent packet metadata, but it does not provide decrypted contents, application bodies, credentials, or a reconstructed TCP stream.

4.3 Metrics and rolling bandwidth

MetricsStore uses synchronized in-memory state to update session totals and time-window measurements. The recent-packet ring buffer is capped at 100 observations; the documented host list can account for up to 1,000 active endpoints, and the alert history is capped at 500 alerts. Rolling buckets are expired as they fall outside the selected observation window.

For a time window of length W seconds, the displayed bandwidth estimate is computed as:

$$\textbf{Bytes per second} = \textbf{bytes observed inside the window} / W$$

$$\textbf{Packets per second} = \textbf{packets observed inside the window} / W$$

Protocol distribution is derived from the counts observed in the current session/window as presented by the relevant chart. Host TX/RX counters are inferred from observed source and destination endpoints on the monitored interface. They describe the traffic visible to the sensor rather than a guaranteed accounting of all traffic on the physical network. For unicast traffic, the same packet can contribute to source TX and destination RX; summing both host sides can therefore count a packet twice.

4.4 Detection algorithms and thresholds

The detection engine uses deterministic heuristics rather than machine learning. Each rule is applied to normalized metadata within a rolling window. These rules indicate suspicious patterns for review; they do not prove malicious intent or confirm that a target has been compromised.

Rule	Observation and grouping	Default trigger	Alert evidence
PORT_SCAN	Bare TCP SYNs grouped by (source IP, destination IP); distinct destination ports are counted.	At least 15 unique destination ports within 10 seconds.	Source and target IP, distinct-port count, sampled destination ports, and timestamps.
SYN_FLOOD	Bare TCP SYN packets grouped by destination IP, aggregating across sources.	At least 100 SYN-without-ACK packets within 5 seconds.	Observed SYN count, time window, and distinct source count; high severity in the documented demo.
ARP_SPOOF	ARP observations grouped by IPv4 claim; the source MAC claims are compared.	Two or more distinct MAC addresses claim the same IPv4 address within 60 seconds.	Claimed IPv4 address and the prior and conflicting MAC addresses.

A bare SYN is represented by SYN=1 and ACK=0. The port-scan detector counts distinct ports rather than total probe packets, limiting inflation from repeat attempts to the same port. The SYN detector groups by the target so that several sources can contribute to a high rate. The ARP detector watches for conflicting address claims. Thresholds are configurable through environment variables for a controlled classroom demonstration.

4.5 Alert lifecycle and evidence

When a detector fires, AlertStore creates a structured alert with a rule identifier, severity, timestamps, and evidence relevant to the trigger. Deterministic fingerprints and a 30-second cooldown deduplicate repeated occurrences: an existing alert can have its last-seen time and occurrence count updated rather than generating an unbounded number of duplicate records. Alerts are bounded in memory and can be acknowledged from the dashboard. Acknowledgement records that an operator reviewed the alert; it does not mean the underlying traffic is confirmed benign.

4.6 API, WebSocket, and frontend

FastAPI exposes operations for service health, interface enumeration, capture status and lifecycle, metric summaries, top hosts, recent packet metadata, alert listing, and alert acknowledgement. The WebSocket endpoint /ws/live publishes compact JSON snapshots at approximately one-second intervals, containing capture status, totals, top hosts, recent packets, and current alerts. The browser also uses REST for initial loading, user actions, and recovery where needed.

The React + TypeScript interface includes an interface picker and capture controls; KPI cards for packets, formatted bytes, packet rate, and active alerts; protocol and throughput charts; a top-talkers table showing TX/RX rates; a bounded recent-packet table with metadata detail; and an alerts panel with severity, evidence details, and acknowledgement. Loading, empty, error, and WebSocket reconnect states are included so that an idle or disconnected sensor is not mistaken for an active one.

4.7 Safety, privacy, and failure handling

- Passive-only runtime: the analyzer observes incoming traffic and does not inject, alter, block, or transmit packets as part of production capture logic.
- Local-only exposure: the backend is configured to bind to 127.0.0.1 by default. It is an unauthenticated educational application and must not be exposed to a public network or untrusted LAN.
- Minimal data retention: packet payloads and PCAP files are not stored; counters, packet rows, detector windows, and alerts remain in bounded memory.
- Explicit user control: capture does not start automatically on backend startup, and interface or permission failures should leave capture stopped with a readable error.
- Safe validation: demonstrations must use only owned/authorized hosts in an isolated VM or Mininet lab. The repository lab utility is documented as refusing non-private destination addresses.

4.8 Development and implementation phases

Phase	Delivered functionality / verification
0. Technical skeleton	Repository structure, FastAPI health endpoint, React/Vite health display, environment setup, and frontend checks.
1. Packet schemas/parser	Normalized packet observation model, protocol-layer parsing, UTC timestamps, and parser unit tests.
2. Metrics	Thread-safe session counters, host TX/RX, rolling bandwidth, bounded recent-packet history, and deterministic tests.
3. Detection and alerts	Three heuristic detectors, threshold windows, bounded alert store, fingerprints/cooldown, acknowledgement, and rule tests.
4. Capture + API	Interface discovery, capture lifecycle, end-to-end packet pipeline, REST routes, and mocked API tests.
5. Live dashboard	WebSocket snapshots, reconnect strategy, KPI cards, charts, tables, evidence view, and frontend type/build verification.
6. Lab validation	Normal baseline, bounded port-scan and SYN-rate scenarios, synthetic ARP conflict validation, and limitations review.
7. Stabilization	Memory-bound checks, session reset checks, local-binding/payload audit, documentation, and final test/build review.

5. Results / Outputs

5.1 Verification summary

The supplied implementation checklist records the phases as completed. Its stabilization entry reports 49 backend tests passing in 0.16 seconds, and the frontend TypeScript check and Vite production build were also verified. Unit tests are designed to run without root privileges or a real network interface by constructing packet fixtures and mocking the relevant API dependencies.

Verification item	Documented result	What it establishes
Backend automated test suite	49 tests passed (reported duration: 0.16 s).	Parser, metrics, detector boundaries/cooldowns, alert storage, API behaviour, and related core logic are covered by automated tests.
Frontend verification	TypeScript check and Vite build passed.	The frontend type-checks and can be bundled as a production build.
Normal traffic baseline	Lab guide states protocol/host counters increment and no alert is raised for the baseline scenario.	Ordinary low-volume traffic should update the dashboard without meeting the three anomaly thresholds.
TCP port-scan scenario	Validated with a bounded Nmap SYN scan in the isolated lab; threshold is 15 unique destination ports in 10 s.	The port-scan rule can generate a PORT_SCAN alert with source/target and port evidence.
SYN-rate scenario	Validated with a bounded SYN-rate test in the isolated lab; threshold is 100 bare SYNs in 5 s.	The detector can generate a SYN_FLOOD alert when the target-level rate threshold is crossed.
ARP conflict scenario	Validated with synthetic in-memory packets.	The detector can flag conflicting MAC claims for the same IPv4 address without poisoning the test machine's ARP cache.

5.2 Detection test matrix

Test case	Input pattern	Expected / documented outcome	Interpretation
T1: Baseline	Low-volume ICMP/ordinary traffic.	Counters update; 0 alerts for the documented baseline.	The normal test pattern stays below the three heuristic thresholds.
T2: Port scan	One source sends bare SYNs to 15 or more distinct destination ports on one target in 10 s.	PORT_SCAN alert; port samples and endpoints shown.	Detects reconnaissance-like port diversity, not attacker identity or intent.
T3: SYN-rate anomaly	At least 100 bare SYNs reach a target in 5 s.	SYN_FLOOD alert with rate evidence and distinct-source count.	Indicates an unusually high arrival rate; it does not prove server resource exhaustion.
T4: ARP conflict	One IPv4 address is claimed by at least two MAC addresses within 60 s.	ARP_SPOOF alert with conflicting address claims.	Highlights a possible spoofing condition; legitimate failover or NIC changes can also cause it.
T5: Parser robustness	Supported layers, unsupported protocols, malformed/missing layers, synthetic timestamps.	Safe classification/fallback; no raw payload in normalized output.	Supports testability and safe processing of packets that do not fit a detector rule.
T6: Alert repetition	Repeated same-rule observations during the cooldown interval.	Existing alert is updated/deduplicated; bounded alert count retained.	Prevents repeated observations from flooding the operator with duplicate alerts.

5.3 Demonstration procedure

Step 1. Prepare the lab. Use a host-only/internal VM network with the analyzer/monitored host at 192.168.56.10 and test client at 192.168.56.20. Verify that the segment is not bridged to public, campus, or production networks.

Step 2. Start the application. Run the FastAPI backend from the backend directory using `uvicorn app.main:app --host 127.0.0.1 --port 8000`. In a second terminal, run `npm run dev` from the frontend directory and open `http://localhost:5173` on the analyzer machine.

Step 3. Begin capture. Select the lab-facing interface from the dashboard and click Start Capture. Provide only the required local packet-capture permissions; verify that the displayed state becomes running.

Step 4. Establish a baseline. From the test client, send a few ping requests to 192.168.56.10. Confirm that packet/protocol and host counters change while the documented low-volume baseline produces no alert.

Step 5. Trigger the port-scan rule. From the test client only, run a bounded SYN scan against ports 1-30 on 192.168.56.10 (for example, with Nmap). The PORT_SCAN rule is expected to fire when at least 15 distinct destination ports are observed within 10 seconds.

Step 6. Validate the SYN-rate and ARP rules. Use the repository's bounded lab utility to generate the documented SYN-rate scenario only against the private VM, and use the provided synthetic ARP-conflict test for ARP detector validation. Do not experiment with these patterns on any shared or external network.

Step 7. Inspect and conclude. Open alert details to review rule identifiers and evidence, acknowledge the alerts, optionally compare bare-SYN metadata in Wireshark using `tcp.flags.syn == 1 && tcp.flags.ack == 0`, and stop capture from the dashboard.

Safety note: the IP addresses above are private-lab examples. The scanning and SYN-rate activities must remain confined to a network that you own and have isolated for this demonstration.

5.4 Dashboard outputs

The implemented dashboard is intended to present the following outputs during a capture session:

- Capture state and monitored interface, with start/stop controls and actionable error states.
- Session packet and byte totals, packet rate, active-alert count, protocol distribution, and rolling traffic/bandwidth trend.
- Top hosts with observed transmit/receive totals and rates to make high-volume endpoints easier to identify.
- Recent packet metadata such as protocol, addresses, ports/flags where available, length, and timestamp; no application payload is presented.
- An alert feed with severity, rule, timestamps, evidence details, occurrence updates, and an acknowledgement action.

5.5 Representative interpretation of an alert

The table below describes the evidence fields that the application is designed to present when a rule fires. It is a rule-to-evidence summary, not a fabricated live packet log or screenshot.

Alert type	Evidence visible to the operator	Meaning and caution
PORT_SCAN	Source IP, target IP, unique destination-port count, sampled ports, time window.	A source has contacted many distinct TCP ports on one target quickly; legitimate inventory/security scans may produce the same pattern.
SYN_FLOOD	Target IP, bare-SYN count, five-second window, distinct source count.	A high rate of connection-initiation packets was observed; the sensor cannot determine from this alone whether the target's SYN backlog was exhausted.
ARP_SPOOF	Claimed IPv4 address, previously observed MAC, conflicting MAC, observation times.	The observed mapping changed within the rule window; redundancy protocols, NIC bonding, VM migration, or normal address changes can also explain it.

5.6 Limitations and result analysis

- Heuristic alerts are indicators for investigation, not proof of a malicious breach. Thresholds can produce false positives when legitimate scanners, stress tests, failovers, or address changes produce similar traffic.
- A passive sensor sees only traffic traversing the selected interface. Asymmetric routing and interface choice may hide part of a conversation or its responses.
- At very high link speeds, a Python callback may not process every packet; the current educational implementation is not a replacement for specialized high-throughput capture systems.
- Encrypted application contents are not inspected. The analyzer uses visible headers and metadata only, and does not decrypt TLS or reconstruct application streams.
- Metrics, packet history, detector windows, and alerts are session-scoped in-memory state. They are reset with a new capture/session or backend restart; no historical database is maintained.
- The API is unauthenticated and local-only. Remote multi-user operation, long-term storage, packet blocking, machine learning, and distributed monitoring are outside the current scope.

6. Conclusion

The Network Traffic Analyzer & Real-Time Anomaly Detector implements a focused, practical application of computer-networking concepts through passive packet observation, protocol-layer parsing, rolling metric aggregation, rule-based anomaly detection, and live browser visualization. Its modular architecture separates capture and parsing from metrics, detection, API delivery, and presentation, while the normalized packet model permits deterministic unit testing independently of elevated capture permissions.

The project implements three explainable detectors: TCP port scans, high target-level rates of bare TCP SYN packets, and conflicting ARP IP-to-MAC claims. Structured evidence, deduplication, cooldowns, acknowledgement, bounded stores, and payload exclusion make the output easier to review and safer to demonstrate. The supplied implementation plan records 49 backend tests passing and successful frontend type/build checks; the lab guide documents baseline and controlled anomaly-validation scenarios.

The results establish a working educational mini IDS, not a complete commercial security product. Its alerts require human interpretation, it only sees traffic on the selected interface, and it does not persist history or actively defend the network. Within those boundaries, the project demonstrates how packet headers, transport flags, address resolution, time windows, client-server APIs, and WebSocket streaming can be combined into a useful local monitoring workflow.

Future scope

Potential future extensions, only after preserving the current safety and test guarantees, include optional export of aggregate metrics, configurable rules through a validated interface, richer false-positive evaluation against labelled traffic, and a queue/worker architecture for higher packet rates. Persistent storage, machine-learning detection, remote deployment, authentication, and active response would require separate threat modelling and requirements rather than being assumed capabilities of this release.

7. References and documentation

[1] Scapy Documentation, “Usage and sniffing.” https://scapy.readthedocs.io/en/latest/usage.html

[2] FastAPI Documentation, “WebSockets” and application lifespan events. https://fastapi.tiangolo.com/advanced/websockets/ ; https://fastapi.tiangolo.com/advanced/events/

[3] React Documentation. https://react.dev/learn

[4] Vite Guide. https://vite.dev/guide/

[5] Mininet Walkthrough. https://mininet.org/walkthrough/

[6] Nmap Reference Guide. https://nmap.org/book/man.html

[7] Wireshark Documentation. https://www.wireshark.org/docs/

[8] RFC 826, “An Ethernet Address Resolution Protocol”; RFC 791, “Internet Protocol”; RFC 8200, “Internet Protocol, Version 6 (IPv6) Specification.”

[9] RFC 768, “User Datagram Protocol”; RFC 793, “Transmission Control Protocol”; RFC 792, “Internet Control Message Protocol”; RFC 4443, “Internet Control Message Protocol (ICMPv6) for the Internet Protocol Version 6 (IPv6) Specification.”

[10] Project materials supplied with this report: README.md, PROJECT_EXPLANATION.txt, LAB_GUIDE.md, IMPLEMENTATION_PLAN.md, and network_traffic_analyzer_prd.md.